# SISTEMAS EMBEBIDOS

## Universidad Politécnica del Bicentenario.

**Monitor de Voltaje y Temperatura con ESP32, Adquisición de Señales Analógicas y Visualización en LCD I²C**

## Ingeniería Mecatrónica 701


Edgar Uriel Rodríguez Moreno *24033660*

Jorge Zamorano Chávez *24033247*

Axel Gabriel Domínguez Macías *24032952*

Braulio Vázquez Morado *24032975*

Daisy Jackeline Hernández Ramos *24033069*


**Silao, Guanajuato a 6 de Noviembre de 2026.**

# Introducción

En el ámbito de la ingeniería de sistemas embebidos, la mecatrónica y la automatización industrial, la adquisición de datos a partir de variables físicas representa uno de los pilares fundamentales para el monitoreo y control de procesos. Los microcontroladores modernos deben ser capaces de procesar señales eléctricas del entorno (como diferencias de potencial, intensidad luminosa o variaciones térmicas) y acondicionarlas digitalmente para su posterior análisis o visualización. La conversión analógica-digital constituye el puente de interconexión entre el mundo físico analógico y el dominio computacional discreto.

El presente reporte documenta el diseño, desarrollo, simulación e implementación práctica de un sistema de instrumentación y adquisición de datos basado en el microcontrolador ESP32 DevKit V1. El objetivo principal consistió en muestrear dos entradas de voltaje variables mediante potenciómetros y estimar la temperatura ambiental a través de un sensor resistivo tipo termistor NTC de 10 kΩ. El procesamiento del sistema requirió la conversión de las lecturas crudas del ADC a valores de tensión real, así como la resolución de modelos matemáticos no lineales para la determinación de la temperatura en grados Celsius. La información procesada se desplegó en tiempo real a través de un monitor serial a 115200 baudios y en una pantalla de cristal líquido LCD 16×2 integrada con el protocolo de comunicación serial I²C, logrando así un sistema embebido robusto, eficiente y autónomo.

# Marco Teórico y Descripción de Componentes

Para la realización de esta práctica se combinaron diversos elementos de hardware y plataformas de software de grado profesional. La tarjeta de desarrollo ESP32 DevKit V1 es una plataforma de procesamiento basada en el SoC ESP32 de Espressif Systems, dotada de una arquitectura de doble núcleo Tensilica Xtensa LX6 de 32 bits, conectividad Wi-Fi y Bluetooth, e integrando periféricos avanzados entre los cuales destaca un conversor analógico-digital con resolución de 12 bits. Su funcionamiento radica en tomar la lectura analógica de los pines GPIO asignados y transformarla en un valor entero en el rango de 0 a 4095 proporcional a la tensión presente en el pin este microcontrolador destaca por su versatilidad de programación, siendo totalmente compatible con C/C++, MicroPython, CircuitPython y el SDK oficial ESP-IDF en lenguaje C.

Como elementos de entrada para la generación de señales eléctricas se utilizaron dos potenciómetros lineales de precisión. Un potenciómetro es un dispositivo pasivo de tres terminales que actúa como un divisor de tensión ajustable manualmente; al conectar sus extremos a la línea de alimentación de 3.3 V y a la tierra común del sistema, la terminal central entrega un voltaje analógico continuo que varía proporcionalmente al ángulo de giro del eje. Aunque los potenciómetros no ejecutan ningún tipo de software al ser componentes electromecánicos pasivos, sus señales son procesadas por cualquier lenguaje o arquitectura con capacidad de lectura analógica.

Para la medición de temperatura se empleó un termistor NTC con una resistencia nominal de 10 kΩ a 25 °C, conectado en serie con una resistencia fija de 10 kΩ para conformar un divisor de voltaje. La resistencia eléctrica de un termistor NTC disminuye exponencialmente a medida que la temperatura del entorno aumenta. Al variar la temperatura, la resistencia del sensor cambia y modifica el voltaje en el punto medio del divisor de tensión. Mediante la Ecuación Beta, el algoritmo implementado en C++ traduce dicho valor de tensión en la resistencia equivalente

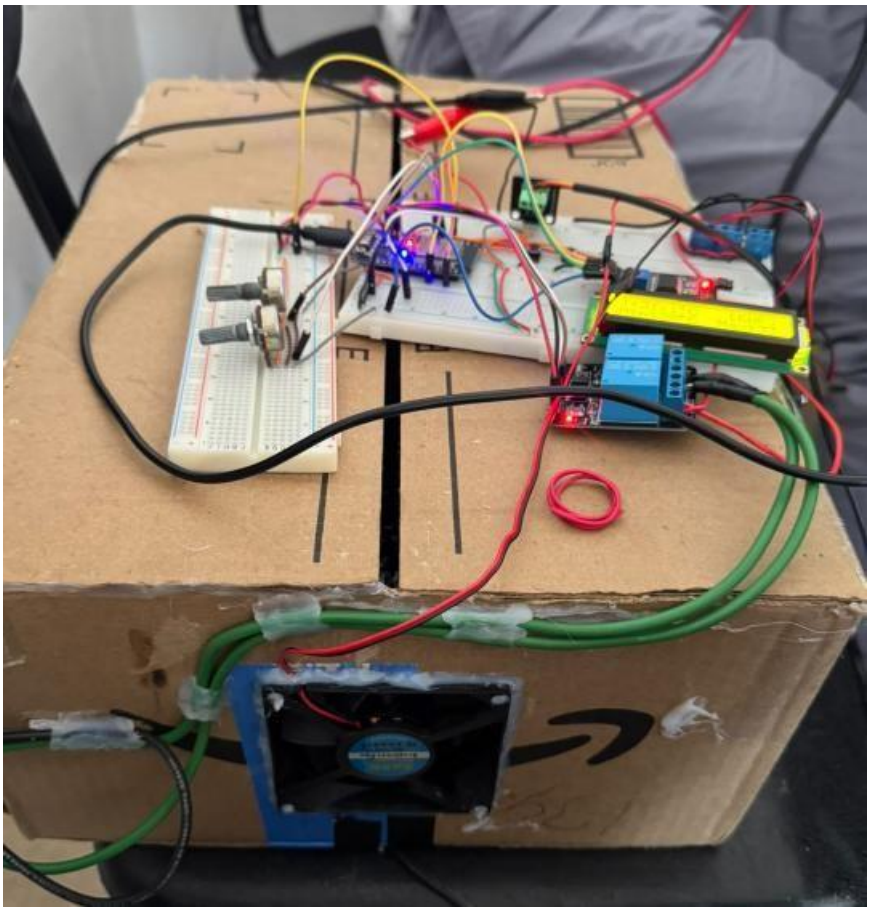
del NTC y, posteriormente, en la temperatura absoluta en Kelvin para su conversión final a grados Celsius.

Para la visualización de los datos se integró una pantalla de cristal líquido de 16 columnas por 2 filas acoplada a un módulo adaptador basado en el circuito integrado PCF8574. Este adaptador permite convertir la comunicación paralela del LCD a un bus de comunicación I²C, reduciendo el número de conexiones físicas necesarias de 6 o más pines digitales a únicamente 2 pines de bus: SDA (datos) y SCL (reloj). La pantalla funciona recibiendo paquetes de datos bajo la dirección de red I²C asignada, permitiendo mostrar texto, caracteres especiales y números en tiempo real mediante bibliotecas de control en C/C++ como LiquidCrystal_I2C o sus equivalentes en Python.

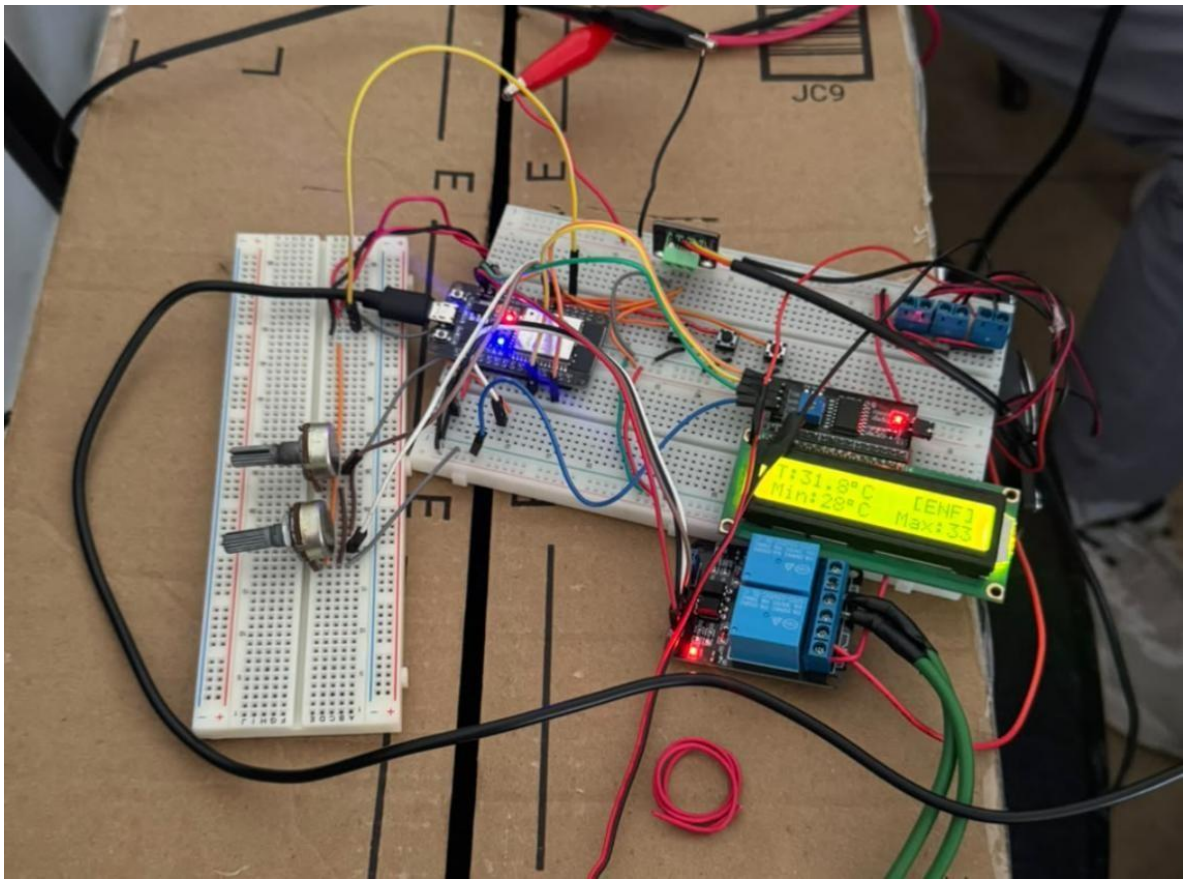
El flujo de trabajo se apoyó en un entorno de desarrollo profesional compuesto por Visual Studio Code como editor principal, la extensión PlatformIO IDE para la gestión de dependencias, compilación e interconexión con el compilador de C++, y el simulador en línea Wokwi para la validación previa del comportamiento electrónico y lógico del circuito, garantizando que el firmware y el esquemático fueran correctos antes del ensamble físico.

## Desarrollo de la Práctica

La ejecución de la práctica comenzó con la preparación del entorno de desarrollo y la configuración inicial de la arquitectura del proyecto dentro de Visual Studio Code utilizando PlatformIO. Se creó un nuevo proyecto destinado a la placa ESP32 DevKit V1 bajo el marco de trabajo de Arduino en C++. En el archivo de configuración platformio.ini se establecieron los parámetros operativos del proyecto, incluyendo la velocidad del puerto serial a 115200 baudios y la inclusión explícita de la dependencia externa para la gestión de la pantalla LCD.



Posteriormente, se realizó el diseño del circuito electrónico y el ensamble de las conexiones físicas y simuladas. Todos los componentes compartieron la misma referencia de voltaje conectándose a la línea de 3.3 V de la ESP32 y a una línea de tierra común para evitar desfases en las lecturas analógicas. El primer potenciómetro se conectó a la terminal GPIO 34, mientras que el segundo potenciómetro se cableó al GPIO 35. Por su parte, la red de medición de temperatura se construyó formando un divisor de tensión entre el termistor NTC de 10 kΩ conectado a la línea de 3.3 V y la resistencia fija de 10 kΩ conectada a GND; la unión central entre ambos componentes se cableó directamente al GPIO 32 para la adquisición de la caída de voltaje en la resistencia fija. Para la pantalla LCD con módulo I²C, las terminales de alimentación se llevaron a las líneas del sistema y los pines de datos y reloj se conectaron a las líneas I²C predeterminadas de la ESP32, es decir, SDA al GPIO 21 y SCL al GPIO 22.




Una vez estructurado el hardware, se desarrolló la lógica del software en el archivo src/main.cpp. En la sección de configuración, se inicializó la comunicación serial a 115200 baudios, se encendió y configuró la pantalla LCD habilitando su luz de fondo y enviando los mensajes de bienvenida. En el ciclo principal, el programa ejecuta lecturas consecutivas del ADC de 12 bits para los pines GPIO 34, 35 y 32, obteniendo valores enteros discretos entre 0 y 4095. A continuación, el firmware aplica la conversión matemática para obtener el voltaje real de cada potenciómetro dividiendo la lectura ADC entre 4095 y multiplicándola por el voltaje de referencia de 3.3 V.

Para el cálculo de la temperatura en el GPIO 32, el programa calcula primero la resistencia instantánea del termistor NTC invirtiendo la fórmula del divisor de voltaje. Con la resistencia calculada, se aplica la Ecuación Beta utilizando la constante nominal, la resistencia de referencia y la temperatura base 25 °C. El resultado obtenido en Kelvin se convierte a grados Celsius restando 273.15.

Finalmente, las tres variables se formatean numéricamente y se imprimen simultáneamente en el monitor serial y en las dos líneas de la pantalla LCD 16×2, agregando un retardo controlado en el bucle para permitir una actualización constante y legible de la información.

```
#include <Arduino.h>
#include <Wire.h>
#include <LiquidCrystal_I2C.h>

// ==========================================
// ASIGNACIÓN DE PINES
// ==========================================
#define LM35_PIN     34   // Sensor de temperatura LM35 (GPIO 34
- ADC1)
#define POT_MIN_PIN  32   // Potenciómetro 1: Ajusta T_min (GPIO
32)
#define POT_MAX_PIN  33   // Potenciómetro 2: Ajusta T_max (GPIO
33)

#define RELE_FOCO    2    // Relé 1: Foco / Calentador (GPIO 2 -
D2)
#define RELE_PELTIER 15   // Relé 2: Peltier / Enfriador (GPIO
15 - D15)

#define LCD_SDA      21   // LCD I2C SDA
#define LCD_SCL      22   // LCD I2C SCL
#define LCD_ADDR     0x27 // Dirección I2C LCD

// Módulo de Relés Active LOW
#define RELE_ON      LOW
#define RELE_OFF     HIGH

// Limites configurados
const int TEMP_RANGO_MIN = 25; // Mínima temperatura elegible
const int TEMP_RANGO_MAX = 60; // Máxima temperatura elegible
// ==========================================
// OBJETOS Y VARIABLES
// ==========================================
LiquidCrystal_I2C lcd(LCD_ADDR, 16, 2);

float temperaturaActual = 0.0;
int tempMin = 25; // Ajustado por Potenciómetro 1 int
tempMax = 60; // Ajustado por Potenciómetro 2

enum EstadoCiclo { CALENTANDO, ENFRIANDO };
EstadoCiclo estadoCiclo = CALENTANDO;

unsigned long ultimaLectura = 0;
const unsigned long INTERVALO_LECTURA = 300; // Refresco cada
300 ms
```

```
// Lectura promediada del LM35 para
estabilidad float leerLM35() {   long sumaADC
= 0;
```

```
  const int MUESTRAS = 30;

  for (int i = 0; i < MUESTRAS; i++) {
sumaADC += analogRead(LM35_PIN);
delayMicroseconds(100);
  }
  float adcPromedio = (float)sumaADC / MUESTRAS;
float milivoltios = (adcPromedio / 4095.0) * 3300.0;
return milivoltios / 10.0; // 10 mV = 1 °C
}

// Lectura de ambos potenciómetros void
leerPotenciometros() {
  int valorPot1 = analogRead(POT_MIN_PIN);
int valorPot2 = analogRead(POT_MAX_PIN);

  // Mapear lecturas entre 25 °C y 60 °C
  tempMin = map(valorPot1, 0, 4095, TEMP_RANGO_MIN,
TEMP_RANGO_MAX);
  tempMax = map(valorPot2, 0, 4095, TEMP_RANGO_MIN,
TEMP_RANGO_MAX);

  // Protección: Asegurar que T_max siempre sea al menos 1°C
superior a T_min
  if (tempMax <= tempMin) {
tempMax = tempMin + 1;
  }
}

// Lógica de control del ciclo del horno (Histéresis) void
controlarCiclo() {
  if (estadoCiclo == CALENTANDO) {
if (temperaturaActual >= tempMax) {
estadoCiclo = ENFRIANDO;
    }
  } else if (estadoCiclo == ENFRIANDO) {
if (temperaturaActual <= tempMin) {
estadoCiclo = CALENTANDO;
    }
  }

  // Control físico de salidas
if (estadoCiclo == CALENTANDO) {
    digitalWrite(RELE_FOCO,  RELE_ON);        //  Encender  Foco
digitalWrite(RELE_PELTIER, RELE_OFF); // Apagar Peltier
  } else {
    digitalWrite(RELE_FOCO, RELE_OFF);   // Apagar Foco
digitalWrite(RELE_PELTIER, RELE_ON);  // Encender Peltier
  }
```

```
}

// Pantalla LCD void
actualizarLCD() {
char linea0[17];
char linea1[17];

  // Línea 0: Temperatura actual del LM35 y estado del ciclo
[CAL/ENF]
snprintf(linea0, sizeof(linea0), "T:%4.1f%cC   [%s]",
temperaturaActual, (char)223, estadoCiclo == CALENTANDO ?
"CAL" : "ENF");

  // Línea 1: Umbral Min (Pot 1) y Umbral Max (Pot 2)
snprintf(linea1, sizeof(linea1), "Min:%2d%cC  Max:%2d%cC",
tempMin, (char)223, tempMax, (char)223);

  lcd.setCursor(0, 0);
lcd.print(linea0);
lcd.setCursor(0, 1);
lcd.print(linea1);
}
void setup() {
  Serial.begin(115200);

  pinMode(LM35_PIN, INPUT);
pinMode(POT_MIN_PIN, INPUT);
pinMode(POT_MAX_PIN, INPUT);

  pinMode(RELE_FOCO, OUTPUT);
pinMode(RELE_PELTIER, OUTPUT);
digitalWrite(RELE_FOCO, RELE_OFF);
digitalWrite(RELE_PELTIER, RELE_OFF);

  Wire.begin(LCD_SDA,
LCD_SCL);   lcd.init();
lcd.backlight();

  lcd.setCursor(0, 0);
lcd.print(" Horno 2 Pots
");   lcd.setCursor(0, 1);
lcd.print(" Iniciando...
");   delay(1500);
lcd.clear();
}
void loop() {
  unsigned long ahora = millis();

  if (ahora - ultimaLectura >= INTERVALO_LECTURA) {
    ultimaLectura = ahora;

    temperaturaActual = leerLM35();
leerPotenciometros();
```

```
controlarCiclo();
actualizarLCD();

    Serial.printf("LM35: %.1f C | T_Min (Pot1): %d C | T_Max
(Pot2): %d C | Estado: %s\n",
                  temperaturaActual, tempMin, tempMax,
estadoCiclo == CALENTANDO ? "CALENTANDO" :
"ENFRIANDO");
  }

}
```

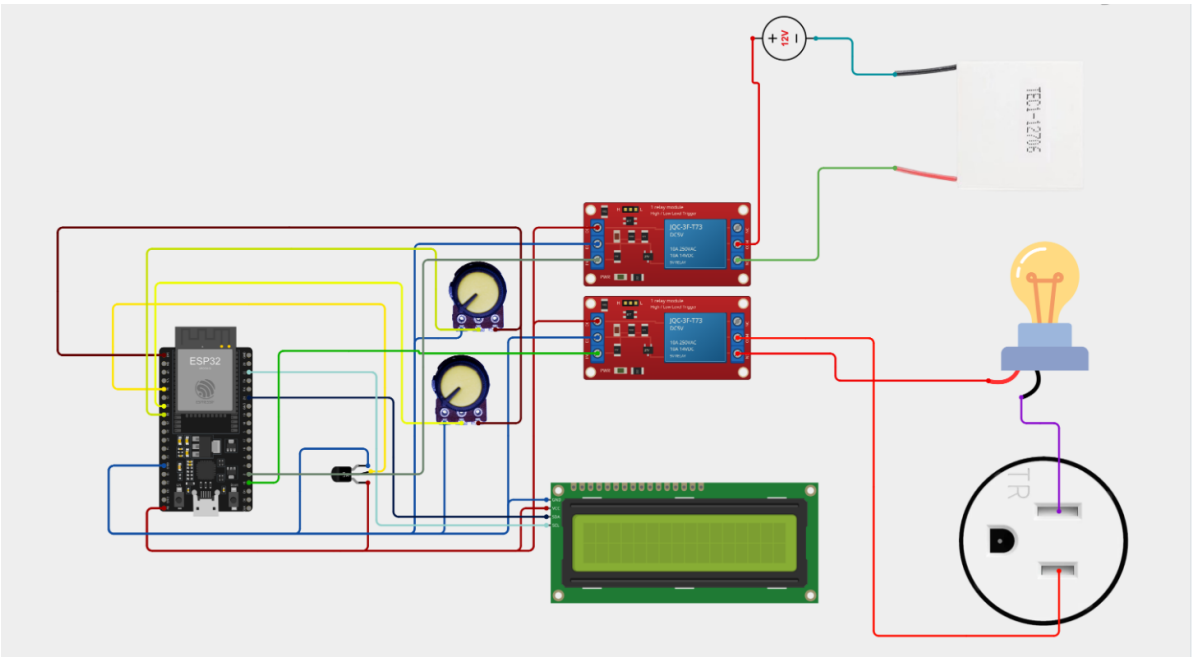


*Diagrama Generado en Github*

## Resultados

El sistema implementado respondió de manera totalmente satisfactoria y estable durante todas las fases de prueba y validación, cumpliendo con la totalidad de los requerimientos de la práctica:

No se detectaron errores de compilación en PlatformIO ni fallas en la transmisión del bus I²C. El sistema arrancó de forma autónoma desplegando correctamente los datos desde el encendido.

**Respuesta de Potenciómetros:** La manipulación manual de los ejes de ambos potenciómetros provocó variaciones inmediatas y lineales en las lecturas de voltaje. Las lecturas en la pantalla y el monitor serial respondieron de manera suave en un rango continuo desde 0.00 V hasta 3.30 V.

**Precisión en la Medición Térmica:** El módulo de estimación de temperatura procesó correctamente los cambios en la resistencia del termistor NTC. Al aplicar calor o variar la temperatura ambiente, la Ecuación Beta calculó de manera adecuada la temperatura ambiental en un rango coherente, mostrando incrementos y decrementos precisos y estables.

**Interfaz de Usuario en Pantalla LCD:** La pantalla LCD 16×2 mostró los datos consolidados en tiempo real sin congelamientos ni parpadeos no deseados. La disposición de la información se mantuvo limpia y ordenada según la siguiente estructura:



**Supervisión mediante Monitor Serial:** La salida enviada al monitor serial a 115200 baudios confirmó la validez de los algoritmos matemáticos ejecutados en el microcontrolador, entregando un flujo continuo de registros de texto que permitieron auditar las lecturas analógicas crudas y sus respectivas equivalencias procesadas.

# Conclusión

El desarrollo de esta práctica permitió validar con éxito la integración de un microcontrolador ESP32 en labores de instrumentación, adquisición de datos y

comunicación serial de periféricos. Se comprobó cuantitativamente el funcionamiento del conversor analógico-digital de 12 bits, así como la efectividad del procesamiento matemático en C++ para transformar lecturas numéricas discretas en variables de tensión real y valores de temperatura mediante el modelo de la Ecuación Beta.

Adicionalmente, se evidenció la ventaja operativa del protocolo I²C, el cual simplifica sustancialmente el diseño físico de circuitos al reducir el número de líneas de conexión requeridas para controlar periféricos de despliegue visual como la pantalla LCD 16×2. Los resultados satisfactorios obtenidos tanto en simulación como en ejecución confirman el dominio de las herramientas modernas de desarrollo embebido y sientan las bases para el diseño de sistemas de monitoreo y control industrial más complejos.

# Referencias


- Boylestad, R. L., & Nashelsky, L. (2018). *Electrónica: Teoría de circuitos y dispositivos electrónicos* (11.ª ed.). Pearson Educación.
- Espressif Systems. (2025). *ESP32 Series Datasheet: Hardware design guidelines and ADC specs*. Espressif Systems Documentation.
- PlatformIO Labs. (2025). *PlatformIO Core & IDE Documentation: Embedded development framework*. Recuperado de https://docs.platformio.org/
- Schwartz, M. (2025). *LiquidCrystal_I2C Library Specification for Arduino/ESP32*. Repositorio oficial en GitHub. Recuperado de https://github.com/marcoschwartz/LiquidCrystal_I2C
- Wokwi. (2025). *Wokwi Simulation Engine: Online ESP32, Sensors and LCD Emulator*. Recuperado de https://wokwi.com/